

Лабораторная работа № 2.3 «Синтаксический анализатор на основе предсказывающего анализа»

20 апреля 2026 г.

Ольга Ивенкова, ИУ9-62Б

Цель работы

Целью данной работы является изучение алгоритма построения таблиц предсказывающего анализатора.

Индивидуальный вариант

```
-- аксиома заключена
-- в фигурные скобки
T, T', { E }, E', F
< E : T E' >
< E' : "+" T E' : @ >
< T : F T' >
< T' : "*" F T' : @ >
< F : "n" : "(" E ")" >
```

Реализация

Неформальное описание синтаксиса входного языка

Входной язык - где сначала идёт список нетерминалов через запятую, причём один из них - аксиома - заключён в фигурные скобки. Потом идут правила в угловых скобках: < noterm1: noterm2 : Если альтернатива пустая, её обозначают через собачку @, и он всегда в конце правила. Нетерминалы - заглавные латинские буквы с возможным штрихом. Терминалы пишутся в двойных кавычках. Комментарии начинаются с – и до конца строки.

Лексическая структура

NONTERM -> [A-Z]’?

TERM -> ”[^\n]*”

EPS -> @

LANGLE -> <

RANGLE -> >

LBRACE -> {

RBRACE -> }

COMMA -> ,

COLON -> :

EOF -> конец ввода

Комментарии: – до конца строки

Грамматика языка

```
Program      -> SymbolList RuleList
SymbolList   -> Symbol SymbolListTail
SymbolListTail -> COMMA Symbol SymbolListTail | e
Symbol       -> NONTERM | LBRACE NONTERM RBRACE

RuleList     -> Rule RuleList | e
Rule         -> LANGLE NONTERM COLON Alternatives RANGLE

Alternatives -> Alternative AlternativesTail
AlternativesTail-> COLON Alternative AlternativesTail | e

Alternative  -> EPS | GrammarSymbol RhsSeq
RhsSeq       -> GrammarSymbol RhsSeq | e
GrammarSymbol -> NONTERM | TERM
```

Программная реализация

```
from __future__ import annotations

from dataclasses import dataclass, field
from typing import ClassVar, Dict, List, Optional, Tuple
import argparse
import sys

@dataclass()
```

```

class Token:
    type: str
    lexeme: str
    line: int
    col: int

class LexError(Exception):
    pass

class ParseError(Exception):
    pass

class Lexer:
    def __init__(self, src: str):
        self.src = src
        self.i = 0
        self.line = 1
        self.col = 1

    def peek(self, k: int = 0) -> str:
        idx = self.i + k
        return self.src[idx] if idx < len(self.src) else "\0"

    def advance(self) -> str:
        ch = self.peek()
        if ch == "\0":
            return ch
        self.i += 1
        if ch == "\n":
            self.line += 1
            self.col = 1
        else:
            self.col += 1
        return ch

    def next_token(self) -> Token:
        while True:
            ch = self.peek()
            if ch == "\0":
                return Token("EOF", "", self.line, self.col)

            if ch.isspace():
                self.advance()
                continue

            if ch == "-" and self.peek(1) == "-":

```

```

        while self.peek() not in ("\0", "\n"):
            self.advance()
        continue

    break

    start_line, start_col = self.line, self.col
    ch = self.peek()

    single = {
        "<": "LANGLE",
        ">": "RANGLE",
        "{": "LBRACE",
        "}": "RBRACE",
        ",": "COMMA",
        ":": "COLON",
        "@": "EPS",
    }

    if ch == "'":
        self.advance()
        buf: List[str] = []
        while True:
            cur = self.peek()
            if cur == "\0" or cur == "\n":
                raise LexError(f"Unterminated string \
                    literal at {start_line}:{start_col}")
            if cur == "'":
                self.advance()
                break
            buf.append(self.advance())
        return Token("TERM", "".join(buf), start_line, start_col)

    if "A" <= ch <= "Z":
        lex = self.advance()
        if self.peek() == "'":
            lex += self.advance()
        return Token("NONTERM", lex, start_line, start_col)

    if ch in single:
        self.advance()
        return Token(single[ch], ch, start_line, start_col)

    raise LexError(f"Unexpected character '{ch}' at {start_line}:{start_col}")

```

```

@dataclass
class Node:
    symbol: str
    label: str = ""
    children: List["Node"] = field(default_factory=list)
    token: Optional[Token] = None
    id: str = field(default_factory=lambda: Node._new_id())

    _counter: ClassVar[int] = 0

    @classmethod
    def _new_id(cls) -> str:
        cls._counter += 1
        return f"n{cls._counter}"

def dot_escape(s: str) -> str:
    return (
        s.replace("\\", "\\")
        .replace("'", "\'")
        .replace("\n", "\\n")
    )

# -----

class Parser:
    TABLE: Dict[str, Dict[str, List[str]]] = {
        "Program": {
            "NONTERM": ["SymbolList", "RuleList"],
            "LBRACE": ["SymbolList", "RuleList"],
        },
        "SymbolList": {
            "NONTERM": ["Symbol", "SymbolListTail"],
            "LBRACE": ["Symbol", "SymbolListTail"],
        },
        "SymbolListTail": {
            "COMMA": ["COMMA", "Symbol", "SymbolListTail"],
            "LANGLE": [],
            "EOF": [],
        },
        "Symbol": {
            "NONTERM": ["NONTERM"],
            "LBRACE": ["LBRACE", "NONTERM", "RBRACE"],
        },
        "RuleList": {

```

```

        "LANGE": ["Rule", "RuleList"],
        "EOF": [],
    },
    "Rule": {
        "LANGE": ["LANGE", "NONTERM", "COLON", "Alternatives", "RANGE"],
    },
    "Alternatives": {
        "EPS": ["Alternative", "AlternativesTail"],
        "NONTERM": ["Alternative", "AlternativesTail"],
        "TERM": ["Alternative", "AlternativesTail"],
    },
    "AlternativesTail": {
        "COLON": ["COLON", "Alternative", "AlternativesTail"],
        "RANGE": [],
    },
    "Alternative": {
        "EPS": ["EPS"],
        "NONTERM": ["GrammarSymbol", "RhsSeq"],
        "TERM": ["GrammarSymbol", "RhsSeq"],
    },
    "RhsSeq": {
        "NONTERM": ["GrammarSymbol", "RhsSeq"],
        "TERM": ["GrammarSymbol", "RhsSeq"],
        "COLON": [],
        "RANGE": [],
    },
    "GrammarSymbol": {
        "NONTERM": ["NONTERM"],
        "TERM": ["TERM"],
    },
}

def __init__(self, tokens: List[Token]):
    self.tokens = tokens
    self.pos = 0
    self.last_alt_was_eps = False

def current(self) -> Token:
    return self.tokens[self.pos]

def syntax_error(self, tok: Token, expected: List[str]) -
> ParseError:
    exp = ", ".join(expected)
    found = tok.type if tok.type != "EOF" else "EOF"
    return ParseError(
        f"Syntax error at {tok.line}:{tok.col}: \

```

```

        expected one of {{{exp}}}, found {found}"
    )

    @staticmethod
    def token_label(tok: Token) -> str:
        if tok.type == "TERM":
            return f'"{tok.lexeme}"'
        if tok.type == "NONTERM":
            return tok.lexeme
        return tok.lexeme

    def parse(self) -> Node:
        root = Node("Program", "Program")
        stack: List[Tuple[str, Optional[Node]]] = [("Program", root)]

        while stack:
            symbol, node = stack.pop()
            tok = self.current()

            if symbol == "EOF":
                if tok.type != "EOF":
                    raise self.syntax_error(tok, ["EOF"])
                self.pos += 1
                continue

            if symbol == "EPS":
                if tok.type != "EPS":
                    raise self.syntax_error(tok, ["EPS"])
                assert node is not None
                node.token = tok
                node.label = "@"
                self.pos += 1
                self.last_alt_was_eps = True
                continue

            if symbol not in self.TABLE:
                if tok.type != symbol:
                    raise self.syntax_error(tok, [symbol])
                assert node is not None
                node.token = tok
                node.label = self.token_label(tok)
                self.pos += 1
                if symbol in {"NONTERM", "TERM"}:
                    self.last_alt_was_eps = False
                continue

```

```

# если уже была @, то новая альтернатива невозможна
if symbol == "AlternativesTail" \
    and tok.type == "COLON" \
    and self.last_alt_was_eps:
    raise ParseError(
        f"Syntax error at {tok.line}:{tok.col}: \
        epsilon alternative must be the last one"
    )

prod = self.TABLE[symbol].get(tok.type)
if prod is None:
    expected = sorted(self.TABLE[symbol].keys())
    raise self.syntax_error(tok, expected)

assert node is not None
node.label = symbol

if symbol == "Rule":
    self.last_alt_was_eps = False

if not prod:
    continue

children = [Node(sym, sym) for sym in prod]
node.children = children

for child in reversed(children):
    stack.append((child.symbol, child))

if self.current().type != "EOF":
    raise self.syntax_error(self.current(), ["EOF"])

return root

# ----- DOT -----
-----

def to_dot(root: Node) -> str:
    lines = [
        "digraph G {",
        '    graph [rankdir=TB];',
        '    node [shape=box, fontname="Courier"];',
    ]
    visited = set()

```

```

def walk(node: Node) -> None:
    if node.id in visited:
        return
    visited.add(node.id)

    lines.append(f'  {node.id} [label=\
        "{dot_escape(node.label or node.symbol)}"]')

    for child in node.children:
        walk(child)
        lines.append(f"  {node.id} -> {child.id}")

    if len(node.children) > 1:
        chain = " -> ".join(ch.id for ch in node.children)
        lines.append(f"  {{ rank=same; {chain} [style=invis] }}")

walk(root)
lines.append("}")
return "\n".join(lines)

def parse_source(src: str) -> Node:
    lexer = Lexer(src)
    tokens: List[Token] = []
    while True:
        tok = lexer.next_token()
        tokens.append(tok)
        if tok.type == "EOF":
            break
    parser = Parser(tokens)
    return parser.parse()

def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("input")
    ap.add_argument("-o", "--output")
    args = ap.parse_args()

    with open(args.input, "r", encoding="utf-8") as f:
        src = f.read()

    try:
        root = parse_source(src)
    except (LexError, ParseError) as e:
        print(str(e), file=sys.stderr)

```

```

        sys.exit(1)

    dot = to_dot(root)
    with open(args.output, "w", encoding="utf-8") as f:
        f.write(dot)

    print(f"OK: дерево вывода записано в {args.output}")

if __name__ == "__main__":
    main()

```

Тестирование

Входные данные

```

-- аксиома заключена
-- в фигурные скобки
T, T', { E }, E', F
< E : T E' >
< E' : "+" T E' : @ >
< T : F T' >
< T' : "*" F T' : @ >
< F : "n" : "(" E ")" >

-- эпсилон обозначен собачкой ;- )

```

Вывод на stdout

```

digraph G {
    graph [rankdir=TB];
    node [shape=box, fontname="Courier"];
    n1 [label="Program"]
    n2 [label="SymbolList"]
    n4 [label="Symbol"]
    n6 [label="T"]
    n4 -> n6
    n2 -> n4
    n5 [label="SymbolListTail"]
    n7 [label=","]
    n5 -> n7
    n8 [label="Symbol"]
    n10 [label="T'"]
    n8 -> n10
    n5 -> n8
    n9 [label="SymbolListTail"]
}

```

```

n11 [label=","]
n9 -> n11
n12 [label="Symbol"]
n14 [label="{"]
n12 -> n14
n15 [label="E"]
n12 -> n15
n16 [label="}"]
n12 -> n16
{ rank=same; n14 -> n15 -> n16 [style=invis] }
n9 -> n12
n13 [label="SymbolListTail"]
n17 [label=","]
n13 -> n17
n18 [label="Symbol"]
n20 [label="E'"]
n18 -> n20
n13 -> n18
n19 [label="SymbolListTail"]
n21 [label=","]
n19 -> n21
n22 [label="Symbol"]
n24 [label="F"]
n22 -> n24
n19 -> n22
n23 [label="SymbolListTail"]
n19 -> n23
{ rank=same; n21 -> n22 -> n23 [style=invis] }
n13 -> n19
{ rank=same; n17 -> n18 -> n19 [style=invis] }
n9 -> n13
{ rank=same; n11 -> n12 -> n13 [style=invis] }
n5 -> n9
{ rank=same; n7 -> n8 -> n9 [style=invis] }
n2 -> n5
{ rank=same; n4 -> n5 [style=invis] }
n1 -> n2
n3 [label="RuleList"]
n25 [label="Rule"]
n27 [label="<"]
n25 -> n27
n28 [label="E"]
n25 -> n28
n29 [label=":" ]
n25 -> n29
n30 [label="Alternatives"]

```

```

n32 [label="Alternative"]
n34 [label="GrammarSymbol"]
n36 [label="T"]
n34 -> n36
n32 -> n34
n35 [label="RhsSeq"]
n37 [label="GrammarSymbol"]
n39 [label="E'"]
n37 -> n39
n35 -> n37
n38 [label="RhsSeq"]
n35 -> n38
{ rank=same; n37 -> n38 [style=invis] }
n32 -> n35
{ rank=same; n34 -> n35 [style=invis] }
n30 -> n32
n33 [label="AlternativesTail"]
n30 -> n33
{ rank=same; n32 -> n33 [style=invis] }
n25 -> n30
n31 [label=">"]
n25 -> n31
{ rank=same; n27 -> n28 -> n29 -> n30 -> n31 [style=invis] }
n3 -> n25
n26 [label="RuleList"]
n40 [label="Rule"]
n42 [label="<"]
n40 -> n42
n43 [label="E'"]
n40 -> n43
n44 [label="."]
n40 -> n44
n45 [label="Alternatives"]
n47 [label="Alternative"]
n49 [label="GrammarSymbol"]
n51 [label="\"+\""]
n49 -> n51
n47 -> n49
n50 [label="RhsSeq"]
n52 [label="GrammarSymbol"]
n54 [label="T"]
n52 -> n54
n50 -> n52
n53 [label="RhsSeq"]
n55 [label="GrammarSymbol"]
n57 [label="E'"]

```

```

n55 -> n57
n53 -> n55
n56 [label="RhsSeq"]
n53 -> n56
{ rank=same; n55 -> n56 [style=invis] }
n50 -> n53
{ rank=same; n52 -> n53 [style=invis] }
n47 -> n50
{ rank=same; n49 -> n50 [style=invis] }
n45 -> n47
n48 [label="AlternativesTail"]
n58 [label=":" ]
n48 -> n58
n59 [label="Alternative"]
n61 [label="@"]
n59 -> n61
n48 -> n59
n60 [label="AlternativesTail"]
n48 -> n60
{ rank=same; n58 -> n59 -> n60 [style=invis] }
n45 -> n48
{ rank=same; n47 -> n48 [style=invis] }
n40 -> n45
n46 [label=">"]
n40 -> n46
{ rank=same; n42 -> n43 -> n44 -> n45 -> n46 [style=invis] }
n26 -> n40
n41 [label="RuleList"]
n62 [label="Rule"]
n64 [label="<"]
n62 -> n64
n65 [label="T"]
n62 -> n65
n66 [label=":" ]
n62 -> n66
n67 [label="Alternatives"]
n69 [label="Alternative"]
n71 [label="GrammarSymbol"]
n73 [label="F"]
n71 -> n73
n69 -> n71
n72 [label="RhsSeq"]
n74 [label="GrammarSymbol"]
n76 [label="T'"]
n74 -> n76
n72 -> n74

```

```

n75 [label="RhsSeq"]
n72 -> n75
{ rank=same; n74 -> n75 [style=invis] }
n69 -> n72
{ rank=same; n71 -> n72 [style=invis] }
n67 -> n69
n70 [label="AlternativesTail"]
n67 -> n70
{ rank=same; n69 -> n70 [style=invis] }
n62 -> n67
n68 [label=">"]
n62 -> n68
{ rank=same; n64 -> n65 -> n66 -> n67 -> n68 [style=invis] }
n41 -> n62
n63 [label="RuleList"]
n77 [label="Rule"]
n79 [label="<"]
n77 -> n79
n80 [label="T'"]
n77 -> n80
n81 [label=":"."]
n77 -> n81
n82 [label="Alternatives"]
n84 [label="Alternative"]
n86 [label="GrammarSymbol"]
n88 [label="\**\"]
n86 -> n88
n84 -> n86
n87 [label="RhsSeq"]
n89 [label="GrammarSymbol"]
n91 [label="F"]
n89 -> n91
n87 -> n89
n90 [label="RhsSeq"]
n92 [label="GrammarSymbol"]
n94 [label="T'"]
n92 -> n94
n90 -> n92
n93 [label="RhsSeq"]
n90 -> n93
{ rank=same; n92 -> n93 [style=invis] }
n87 -> n90
{ rank=same; n89 -> n90 [style=invis] }
n84 -> n87
{ rank=same; n86 -> n87 [style=invis] }
n82 -> n84

```

```

n85 [label="AlternativesTail"]
n95 [label=":"]
n85 -> n95
n96 [label="Alternative"]
n98 [label="@"]
n96 -> n98
n85 -> n96
n97 [label="AlternativesTail"]
n85 -> n97
{ rank=same; n95 -> n96 -> n97 [style=invis] }
n82 -> n85
{ rank=same; n84 -> n85 [style=invis] }
n77 -> n82
n83 [label=">"]
n77 -> n83
{ rank=same; n79 -> n80 -> n81 -> n82 -> n83 [style=invis] }
n63 -> n77
n78 [label="RuleList"]
n99 [label="Rule"]
n101 [label="<"]
n99 -> n101
n102 [label="F"]
n99 -> n102
n103 [label=":"]
n99 -> n103
n104 [label="Alternatives"]
n106 [label="Alternative"]
n108 [label="GrammarSymbol"]
n110 [label="\n\n"]
n108 -> n110
n106 -> n108
n109 [label="RhsSeq"]
n106 -> n109
{ rank=same; n108 -> n109 [style=invis] }
n104 -> n106
n107 [label="AlternativesTail"]
n111 [label=":"]
n107 -> n111
n112 [label="Alternative"]
n114 [label="GrammarSymbol"]
n116 [label="\n(\n"]
n114 -> n116
n112 -> n114
n115 [label="RhsSeq"]
n117 [label="GrammarSymbol"]
n119 [label="E"]

```

```

n117 -> n119
n115 -> n117
n118 [label="RhsSeq"]
n120 [label="GrammarSymbol"]
n122 [label="\")\""]
n120 -> n122
n118 -> n120
n121 [label="RhsSeq"]
n118 -> n121
{ rank=same; n120 -> n121 [style=invis] }
n115 -> n118
{ rank=same; n117 -> n118 [style=invis] }
n112 -> n115
{ rank=same; n114 -> n115 [style=invis] }
n107 -> n112
n113 [label="AlternativesTail"]
n107 -> n113
{ rank=same; n111 -> n112 -> n113 [style=invis] }
n104 -> n107
{ rank=same; n106 -> n107 [style=invis] }
n99 -> n104
n105 [label=">"]
n99 -> n105
{ rank=same; n101 -> n102 -> n103 -> n104 -> n105 [style=invis] }
n78 -> n99
n100 [label="RuleList"]
n78 -> n100
{ rank=same; n99 -> n100 [style=invis] }
n63 -> n78
{ rank=same; n77 -> n78 [style=invis] }
n41 -> n63
{ rank=same; n62 -> n63 [style=invis] }
n26 -> n41
{ rank=same; n40 -> n41 [style=invis] }
n3 -> n26
{ rank=same; n25 -> n26 [style=invis] }
n1 -> n3
{ rank=same; n2 -> n3 [style=invis] }
}

```

Вывод

Изучила алгоритм построения таблиц предсказывающего анализатора.